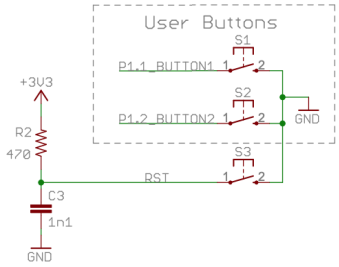
EEL 4742C: Embedded Systems
Name: Ryan Ramdihal
Lab 2: Using the Push Buttons

Introduction:

In Lab 2, we will be utilizing the push buttons on the MSP430F6989 launchpad board, to implement solutions for real life applications, like parking space sensors or machines overloading power. The first objective of the lab is to configure the ports of the buttons and use masking operations to turn the LED on and off depending on if certain buttons are pushed or not. After configuration of the necessary button ports, we set an if statement that if the S1 button is being pressed the red LED will turn on. We then enhance the code by adding a second button, which turns on the green LED, with the intention that if both buttons are pressed, both LED lights turn on. The next intention is to implement an industrial application where both LEDs cannot be on at the same time, even if both buttons are pressed. The system follows a "first come, first serve" approach. The second button's LED remains inactive until the first button is released, and vice versa.The final objective in this lab is to implement a kill switch that resets the LEDs if both buttons are pressed concurrently. In this scenario, both buttons must be released before any further action can be initiated, and thus completes this lab.

Part 2.1: Reading the Push Buttons

The approach initiates by identifying and configuring pins associated with the buttons, followed by their mapping to the LEDs. The LEDs are set to output, while the buttons are designated as input. Notably, when a button is pressed. it registers as 0, signifying the button is active low. To address this, a pull up resistor is used since P1OUT is not used for data. Refining the provided pseudocode, the condition is if P1IN and BUT1 are equal to 0, indicating that the button S1 is pressed, triggering the instruction for the red LED to turn on. Conversely, if the condition is not met, the S1 button released, the red LED is turned off. Accomplishing the first objective of this lab.



2.1 Code:

```
#include <msp430fr6989.h>
#define redLED BIT0 // Red LED at P1.0
#define greenLED BIT7 // Green LED at P9.7
#define BUT1 BIT1 // Button S1 at P1.1
#define BUT2 BIT2 // Button S2 at P1.2
void main(void) {
WDTCTL = WDTPW | WDTHOLD; // Stop the Watchdog timer
```

```
PM5CTL0 &= ~LOCKLPM5; // Enable the GPIO pins
// Configure and initialize LEDs
P1DIR |= redLED; // Direct pin as output
P9DIR |= greenLED; // Direct pin as output
P1OUT &= ~redLED; // Turn LED Off
P9OUT &= ~greenLED; // Turn LED Off
// Configure buttons
P1DIR &= ~BUT1; // Direct pin as input
P1REN |=BUT1; // Enable built-in resistor
P1OUT |=BUT1; // Set resistor as pull-up
P1DIR &= ~BUT2;
P1REN |=BUT2; // Enable built-in resistor
P1OUT |=BUT2; // Set resistor as pull-up
// Polling the button in an infinite loop
for(;;) {
// Rewrite the pseudocode below into C code
if ( (P1IN &BUT1) == 0 ) //if button pressed turn on
{
 P1OUT ^= redLED;
}
else{
   P1OUT &= ~redLED; //if button released turn off
}
}
}
```

Part 2.2: Two Push Buttons: Occupancy Monitor

In extending our design from the previous objective, aiming to adapt the existing code to simulate a parking spot sensor system. The objective is for the designated LED to turn on when its corresponding button is pressed, symbolizing the occupation of a parking spot sensor system. To achieve this, we replicate the configuration for button S1 from the initial code to button S2. Integrating the second button and its associated green LED, we augment the code with an additional if-else statement. Specifically, we modify the condition to check if both P1IN and BUT2 are equal to 0, indicating the pressing of button S2. Within the if statement to code directs the green LED to turn on. Conversely, the else statement ensures that the green LED is turned off, thus representing the parking spot as unoccupied. This adaption expands the applicability to the analogy of handling multiple parking spots.

2.2 Code:

```
#include <msp430fr6989.h>
#define redLED BIT0 // Red LED at P1.0
#define greenLED BIT7 // Green LED at P9.7
#define BUT1 BIT1 // Button S1 at P1.1
#define BUT2 BIT2 // Button S2 at P1.2
void main(void) {
WDTCTL = WDTPW | WDTHOLD; // Stop the Watchdog timer
PM5CTL0 &= ~LOCKLPM5; // Enable the GPIO pins
// Configure and initialize LEDs
P1DIR |= redLED; // Direct pin as output
```

```
P9DIR |= greenLED; // Direct pin as output
P1OUT &= ~redLED; // Turn LED Off
P9OUT &= ~greenLED; // Turn LED Off
// Configure buttons
P1DIR &= ~BUT1; // Direct pin as input
P1REN |=BUT1; // Enable built-in resistor
P1OUT |=BUT1; // Set resistor as pull-up
P1DIR &= ~BUT2;
P1REN |=BUT2; // Enable built-in resistor
P1OUT |=BUT2; // Set resistor as pull-up
// Polling the button in an infinite loop
for(;;) {
// Rewrite the pseudocode below into C code
if ( (P1IN &BUT1) == 0 ) //if pressed turn on
{
 P1OUT ^= redLED;
}
else{
   P1OUT &= ~redLED; // turn off if not pressed
}
if((P1IN &BUT2) == 0) //if pressed turn on
{
   P9OUT ^= greenLED;
}
else
{
   P9OUT &= ~greenLED; // turn off if not pressed
}
}
}
```

Part 2.3: Two Push Buttons: Electric Generator Load Control (v1)

The next objective involves simulating an industrial application where two machine operators request to use their machines by pressing/holding their designated buttons. The constraint being that both machines cannot operate simultaneously to avoid a power overload. In the event that both buttons are pressed, the program is to prioritize the first button pressed, keeping its LED on until released. Only then the other button, if pressed, activates with its corresponding LED, ensuring exclusive usage of one machine at a given moment. To implement this logic, I incorporated if statements to handle the basic scenario if either button is not pressed their LED will be set to off. Subsequently, I added two additional if statements. The first checks if button 1 is pressed AND button 2 is not pressed, instructing the board to light the LED associated with button 1. The second if statement reflects the same, but for button 2. By incorporating the AND operator in these if statement conditions, the system ensures that only when a single button is pressed, the corresponding LED will be activated, adhering to the constraints of this application.

2.3 Code:

```
#include <msp430fr6989.h>
#define redLED BIT0 // Red LED at P1.0
#define greenLED BIT7 // Green LED at P9.7
```

```
#define BUT1 BIT1 // Button S1 at P1.1
#define BUT2 BIT2 // Button S2 at P1.2
void main(void) {
   WDTCTL = WDTPW | WDTHOLD; // Stop the Watchdog timer
   PM5CTL0 &= ~LOCKLPM5; // Enable the GPIO pins
   // Configure and initialize LEDs
   P1DIR |= redLED; // Direct pin as output
   P9DIR |= greenLED; // Direct pin as output
   P1OUT &= ~redLED; // Turn LED Off
   P9OUT &= ~greenLED; // Turn LED Off
   // Configure buttons
   P1DIR &= ~BUT1; // Direct pin as input
   P1REN |= BUT1; // Enable built-in resistor
   P1OUT |= BUT1; // Set resistor as pull-up
   P1DIR &= ~BUT2;
   P1REN |= BUT2; // Enable built-in resistor
   P1OUT |= BUT2; // Set resistor as pull-up
   // Polling the button in an infinite loop
   for (;;) {
      //if button 1 not pressed turn off led
      if ((P1IN & BUT1) != 0) {
         P1OUT &= ~redLED;
      }
      //if button 2 not pressed turn off led
      if ((P1IN & BUT2) != 0) {
         P9OUT &= ~greenLED;
      }
      //if button 1 pressed and button 2 not pressed turn on red
      if (((P1IN & BUT1) == 0) && ((P1IN & BUT2) != 0)) {
         P1OUT |= redLED;
      }
      //if button 2 pressed and button 1 not pressed turn on green
      if (((P1IN & BUT2) == 0) && ((P1IN & BUT1) != 0)) {
         P9OUT |= greenLED;
      }
   }
}
```

Part 2.4: Two Push Buttons: Electric Generator Load Control (v2)

To optimize the system further, the goal is to implement a safety feature, a kill switch. This feature ensures that if both buttons are pressed simultaneously, the associated LED/ machine immediately powers off. The machine cannot restart until both buttons have been released guaranteeing the operation of only one machine at a time. The modification I provided was from incorporating a new while loop positioned ahead of the existing if statements. This new while loop activates when both buttons are pressed simultaneously, this condition is set to trap the iterations if both buttons are pressed, stopping any ability for an LED to be turned on.This prevents any starting of the machinery until the safe state is restored. Inside this loop, I included an if statement where it can break out of the loop only if both buttons are released, restoring the ability of turning on a machine/LED by the other if statements outside that while loop. Following this feature, the board can resume normal and secure operation allowing one machine to function at a time.

2.4 Code:

```
#include <msp430fr6989.h>
#define redLED BIT0 // Red LED at P1.0
#define greenLED BIT7 // Green LED at P9.7
#define BUT1 BIT1 // Button S1 at P1.1
#define BUT2 BIT2 // Button S2 at P1.2
void main(void) {
WDTCTL = WDTPW | WDTHOLD; // Stop the Watchdog timer
PM5CTL0 &= ~LOCKLPM5; // Enable the GPIO pins
// Configure and initialize LEDs
P1DIR |= redLED; // Direct pin as output
P9DIR |= greenLED; // Direct pin as output
P1OUT &= ~redLED; // Turn LED Off
P9OUT &= ~greenLED; // Turn LED Off
// Configure buttons
P1DIR &= ~BUT1; // Direct pin as input
P1REN |=BUT1; // Enable built-in resistor
P1OUT |=BUT1; // Set resistor as pull-up
P1DIR &= ~BUT2;
P1REN |=BUT2; // Enable built-in resistor
P1OUT |=BUT2; // Set resistor as pull-up
// Polling the button in an infinite loop
for(;;) {
// Rewrite the pseudocode below into C code
if((P1IN &BUT1) == 0 && ((P1IN &BUT2) == 0)){ //if both buttons are pressed
   for(;;)
   {
      if((P1IN &BUT1) != 0 && ((P1IN &BUT2) != 0)) //if both buttons are released
      {
         break;
      }
   }
}
if ( (P1IN &BUT1) == 0 && ((P1IN &BUT2) != 0)) //turns only one when that button is pressed
{
 P1OUT ^= redLED;
}
else{
   P1OUT &= ~redLED;
}
if((P1IN &BUT2) == 0 && ((P1IN &BUT1) != 0))
{
   P9OUT ^= greenLED;
}
else
{
   P9OUT &= ~greenLED;
}
}
}
```

Conclusion:

Lab 2 provided valuable insights into utilizing the push buttons on the MSP430FR6989 launchpad board to address applications, such as parking sensors and machines at risk of power overload. Our initial objective was complete after configuring button ports and employing masking operations to control the LED states based on button inputs. Adding a layer of complexity, introducing a second button, illuminating the green LED in tandem with the red LED when both buttons are pressed. Transitioning to an industrial application, implementing logic that ensures that both LEDs cannot be active simultaneously. The "first come, first serve" approach adds a practical dimension to the system, wherein the second button's LED remains inactive until the first button is released, and vice versa. The culmination of our efforts in Lab 2 involved implementing a kill switch. The mechanism resets both LEDs if both buttons are pressed concurrently. The requirement for both buttons to be released before further action emphasizes the importance of controlled operations, making our system robust and responsive in various real-world scenarios. Overall, Lab 2 has provided a comprehensive hands-on experience in developing effective solutions for practical applications using embedded systems.

Student Q&A

1. When a pin is configured as input, P1IN is used for data, which leaves P1OUT available for other use? In such a case, what is P1OUT used for?

Used to configure the built in resistors if P1OUT is not used for data.

2. A programmer wrote this line of code to check if bit 3 is equal to 1: if((Data & BIT3)==1). Explain why this if-statement is incorrect.

The if statement should be if ((Data AND BIT3) != 0) because equaling it to 1 does not check if the specific BIT3 mask is set in the Data variable.

3. Comment on the codes' power-efficiency if the device is battery operated. Is reading the button via polling power efficient?

No, because continuously checking the state of the button will drain its power, but by the use of interrupts, a microcontroller can be triggered only when necessary, conserving its energy and battery life.